

1 DISPLAY

display controls how an element participates in the page layout.

Think of it as the base personality of an element.

✓ Block elements

- Take full width of the parent.
- Always start on a new line.
- Can set width, height, margin, padding.

Examples: div, p, h1–h6, section.

Example

```
<div class="block">Block A</div>
```

```
<div class="block">Block B</div>
```

```
.block {  
  display: block;  
  background: #dfe8ff;  
  margin: 10px 0;  
  padding: 10px;  
}
```

✓ Good for stacking content vertically.

✓ Inline elements

- Only take space equal to their content.
- Do not start on a new line.
- Cannot set width/height.
- Respect left/right margins but not top/bottom.

Examples: span, a, strong.

Example

```
<p>
```

```
  This is <span class="inline">inline</span> text.
```

```
</p>
```

✓ Inline-block

- Behaves like inline (appears in same line)
- But can set width & height like block.

Example (buttons)

```
<a class="btn">Login</a>
<a class="btn">Register</a>
.btn {
  display: inline-block;
  padding: 8px 12px;
  background: #4caf50;
  color: #fff;
  border-radius: 5px;
}
```

✓ Perfect for button-like elements.

✓ Display: none vs visibility: hidden

display: none; /* Element removed from layout */

visibility: hidden; /* Element space reserved but invisible */

☆ Teaching Suggestion

Tell students:

"If layout breaks, first check the display property."

2 POSITION

position determines how an element is placed relative to the page or another element.

✓ 1. static (default)

- The normal flow of the page.
- No positioning offsets (top, left, etc.) work here.

✓ 2. relative

- Stays in normal flow but can be shifted.
- Useful for creating a positioning context for children.

Example

```
<div class="box">Move me</div>
```

```
.box {  
  position: relative;  
  top: 10px;  
  left: 20px;  
  background: #f4c2c2;  
}
```

✓ 3. absolute

- Removed from document flow.
- Positioned relative to the nearest positioned ancestor.

Example: Tooltip

```
<div class="btn-wrapper">  
  Hover me  
  <span class="tooltip">Tooltip Text</span>  
</div>
```

```
.btn-wrapper {  
  position: relative;  
  padding: 20px;  
  background: #eee;  
}
```

```
.tooltip {  
  position: absolute;  
  top: -30px;  
  left: 0;  
  background: black;  
  color: white;  
  padding: 5px;  
  border-radius: 3px;  
}
```

✓ The tooltip appears over the parent because of position: relative on parent.

✓ 4. fixed

- Stays in the viewport, even during scrolling.

Example: Fixed navbar

```
<header class="nav">My Website</header>
```

```
.nav {  
  position: fixed;  
  top: 0;  
  width: 100%;  
  background: #333;  
  padding: 15px;  
  color: #fff;  
}
```

- ✓ Perfect for sticky headers, chat buttons, floating icons.

✓ 5. sticky

- Behaves like relative until a scroll threshold, then sticks.

Example

```
.sticky-title {  
  position: sticky;  
  top: 0;  
  background: white;  
  padding: 10px;  
}
```

☆ Teaching Suggestion

Ask:

“What happens if we remove position: relative from the parent of an absolute element?”

Students will see the child sticks to the entire page instead!

3 FLOAT

Float was originally meant for wrapping text around images but was misused for layout for years.

Now replaced mostly by flexbox/grid — but still important to understand.

✓ Example: Text wrap

```

<p>
  Lorem ipsum... text will wrap around the image on the left.
</p>
```

```
.left-img {
  float: left;
  margin: 0 15px 10px 0;
}
```

✓ Good for newspaper-style layouts, wrapping images.

✓ Float layout issue (parent collapse)

Floating child elements cause parent height to collapse.

Example fix: Clearfix

```
.clearfix::after {
  content: "";
  display: block;
  clear: both;
}
```

Use like:

```
<div class="container clearfix">
  <div class="left"></div>
  <div class="right"></div>
</div>
```

☆ Teaching Suggestion

Explain:

“Float is not a layout system. It’s for wrapping content.”

Modern layouts → Flexbox, Grid.

4 FLEXBOX

(Modern, powerful layout system)

Flexbox makes it easy to create responsive, aligned, centered, spaced layouts without hacks.

☆ Flexbox Core Ideas

Turn on flex:

```
display: flex;
```

Control direction:

```
flex-direction: row | column;
```

Control alignment:

```
justify-content: center;
```

```
align-items: center;
```

Make items flexible:

```
flex: 1;
```

✓ Example 1 — Basic horizontal layout

```
<div class="flexbox">  
  <div class="item">A</div>  
  <div class="item">B</div>  
  <div class="item">C</div>  
</div>
```

```
.flexbox {  
  display: flex;  
  gap: 10px;  
  background: #eee;  
  padding: 15px;  
}
```

```
.item {  
  background: #aedaf7;  
  padding: 20px;  
}
```

✓ Example 2 — Centering perfectly

```
<div class="parent">  
  <div class="child">CENTERED</div>  
</div>
```

```
.parent {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  height: 250px;  
  background: #ddd;  
}
```

```
.child {  
  background: #ff6b81;  
  padding: 20px 40px;  
  color: white;  
}
```

✓ This is the most common real-world use.

✓ Example 3 — Responsive Cards

```
<div class="cards">  
  <div class="card">Card 1</div>  
  <div class="card">Card 2</div>  
  <div class="card">Card 3</div>  
</div>
```

```
.cards {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 15px;  
}  
.card {  
  flex: 1 1 200px;  
  background: #fff;  
  padding: 20px;  
  border: 1px solid #ccc;  
}
```

✓ Cards adjust to screen size automatically.

✓ Example 4 — Navigation Bar

```
<nav class="nav">
  <div class="logo">LOGO</div>
  <ul class="menu">
    <li>Home</li>
    <li>About</li>
    <li>Services</li>
    <li>Contact</li>
  </ul>
</nav>
```

```
.nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 15px 25px;
  background: #222;
  color: #fff;
}
```

```
.menu {
  display: flex;
  gap: 20px;
  list-style: none;
}
```

✓ A real-world use case for flexbox.